

RxJS Operator Reference — Interview-Ready Notes

1. Higher-Order Mapping Operators

Higher-order mapping operators take a value from a source Observable, create an **inner Observable**, and then flatten the resulting `Observable<Observable<T>>`.

The four important operators are:

Operator	New source value arrives	Concurrent?	Order preserved?	Typical use case
<code>switchMap</code>	Unsubscribes from previous inner Observable	✗	Latest only	Search/typeahead
<code>mergeMap</code>	Subscribes to every inner Observable	✓	✗	Independent parallel operations
<code>concatMap</code>	Queues new inner Observables	✗	✓	Ordered saves/uploads
<code>exhaustMap</code>	Ignores new values while busy	✗	First wins	Login/submit button

Memory Trick

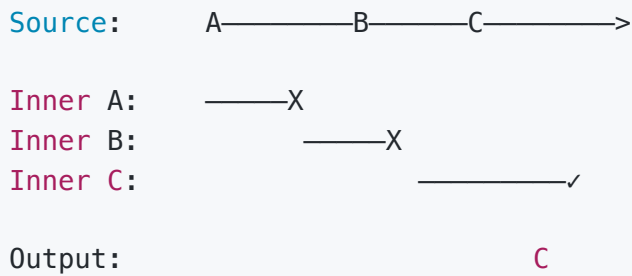
S-M-C-E

Switch → latest Merge → many Concat → queue Exhaust → ignore

1.1 `switchMap` — Latest Wins

`switchMap` unsubscribes from the previous inner Observable when a new source value arrives.

Flow



Only the latest active inner Observable is allowed to continue.

Example — Search

```
import { fromEvent, of } from 'rxjs';
import {
  debounceTime,
  distinctUntilChanged,
  map,
  switchMap
} from 'rxjs/operators';

const search$ = fromEvent<InputEvent>(
  document.getElementById('search')!,
  'input'
).pipe(
  map(event => (event.target as HTMLInputElement).value),
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(query =>
    searchApi(query)
  )
);

search$.subscribe(results => {
  console.log(results);
});
```

Conceptually:

```
const searchApi = (query: string) =>
  http.get(`/api/search?q=${query}`);
```

Why `switchMap` ?

Suppose:

User types:

a
an
ang
angu
angul
angular

You don't want the response for "a" to overwrite the response for "angular".

`switchMap` ensures that older subscriptions are unsubscribed when a newer query arrives.

Important distinction

`switchMap` **cancels/unsubscribes from the previous inner Observable**. It does not universally guarantee that the underlying operation itself is physically cancelled.

With Angular `HttpClient`, unsubscribing from an in-flight request generally causes the underlying HTTP request to be aborted.

Trade-offs

Advantages

- Prevents stale results.
- Excellent for typeahead/search.
- Prevents unnecessary continued processing of previous inner streams.

Disadvantages

- Previous work can be discarded.
- Wrong choice when **every request must complete**.

Avoid `switchMap` when

Every operation must complete

↓

Don't **use** `switchMap`

↓

Consider `mergeMap` / `concatMap`

1.2 mergeMap — Everything Runs

`mergeMap` subscribes to every inner Observable immediately, allowing concurrent execution.

Flow

Source: A—B—C—>

Inner A: _____✓

Inner B: _____✓

Inner C: _____✓

Output: B—A—C

The output order depends on completion timing.

Example

```
import { from } from 'rxjs';
import { mergeMap } from 'rxjs/operators';

const saveUser = (id: number) =>
  api.saveUser(id);

from([1, 2, 3]).pipe(
  mergeMap(id => saveUser(id))
).subscribe(result => {
  console.log(result);
});
```

All three operations can execute concurrently.

Concurrency control

A major interview point:

```
mergeMap(project, concurrent)
```

Example:

```
from([1, 2, 3, 4, 5]).pipe(  
  mergeMap(  
    id => saveUser(id),  
    2  
  )  
);
```

At most **2 inner Observables** execute concurrently.

Use cases

- Independent API calls
- Delete multiple records
- Upload independent files
- Event processing
- Parallel processing

Trade-offs

Advantages

- High throughput.
- Parallel execution.
- Does not cancel previous work.
- Supports concurrency limits.

Disadvantages

- Output ordering is not guaranteed.
- Too much concurrency can overload APIs/resources.
- Long-lived inner Observables can keep subscriptions alive.

1.3 concatMap — Queue and Preserve Order

`concatMap` waits for the current inner Observable to complete before subscribing to the next one.

Flow



The next operation starts only after the previous one completes.

Example

```
import { from } from 'rxjs';
import { concatMap } from 'rxjs/operators';

const saveUser = (id: number) =>
  api.saveUser(id);

from([1, 2, 3]).pipe(
  concatMap(id => saveUser(id))
).subscribe(result => {
  console.log(result);
});
```

Execution:

```
saveUser(1)
  ↓
complete
  ↓
saveUser(2)
  ↓
complete
  ↓
saveUser(3)
```

Use cases

- Ordered database updates
- Sequential file uploads
- Sequential API operations
- Operations where order matters

Trade-offs

Advantages

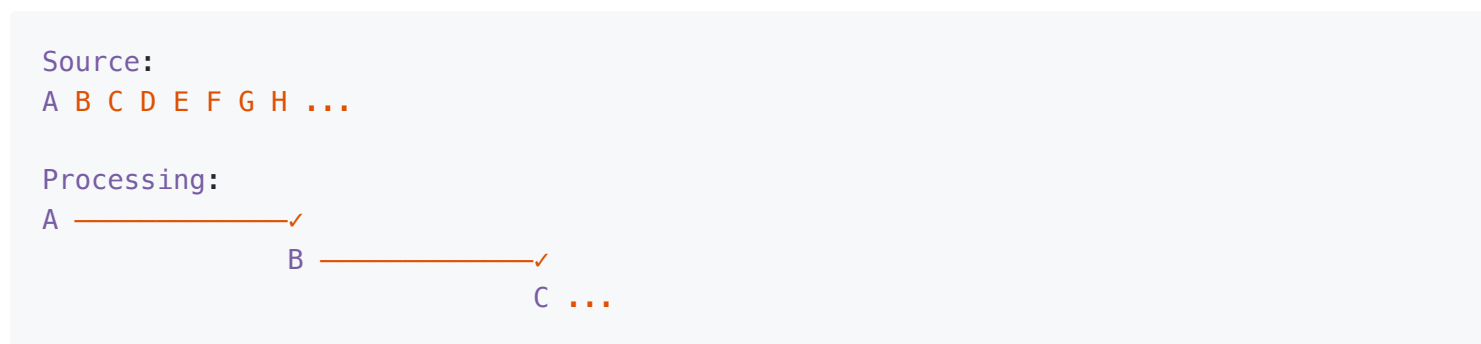
- Guarantees ordering.
- No concurrent inner execution.
- Prevents race conditions caused by ordering.

Disadvantages

- Slower than `mergeMap`.
- A slow request blocks all subsequent requests.
- Source emissions can accumulate in the queue.

Important interview point

If the source emits faster than the inner Observable completes:

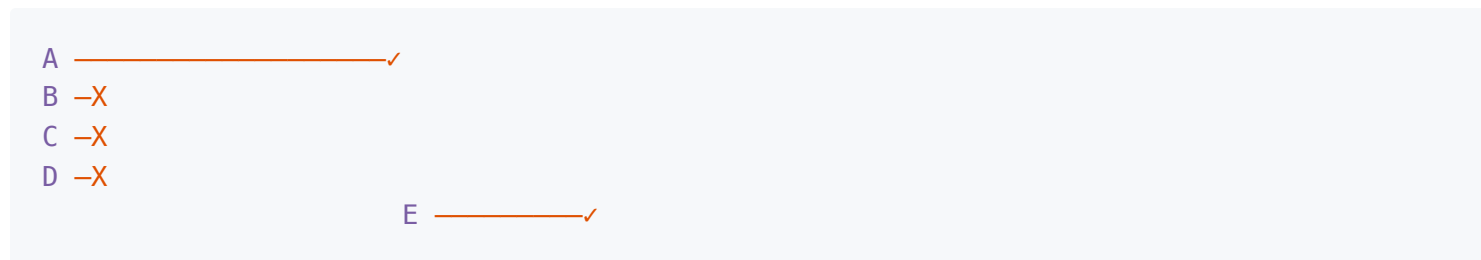


The pending values can accumulate.

1.4 `exhaustMap` — First Wins While Busy

`exhaustMap` subscribes to the first inner Observable and ignores subsequent source emissions until it completes.

Flow



Example — Submit button

```
import { fromEvent } from 'rxjs';
import { exhaustMap } from 'rxjs/operators';

const submit$ = fromEvent(
  document.getElementById('submit')!,
  'click'
);

submit$.pipe(
  exhaustMap(() => submitForm())
).subscribe(result => {
  console.log('Submitted:', result);
});
```

If the user clicks:

```
Click 1 → API starts
Click 2 → ignored
Click 3 → ignored
Click 4 → ignored
API completes
Click 5 → API starts
```

Use cases

- Login
- Payment submission
- Form submission
- Prevent duplicate button clicks
- One-at-a-time operations

Trade-offs

Advantages

- Prevents duplicate concurrent operations.
- Simple way to protect non-repeatable actions.

Disadvantages

- User actions can be silently ignored.
- Not appropriate when every event must be processed.

Higher-Order Operator Decision Tree



One-line interview answer

`switchMap` cancels the previous subscription, `mergeMap` runs inner Observables concurrently, `concatMap` queues them sequentially, and `exhaustMap` ignores new source values while an inner Observable is active.

2. Combination Operators

Combination operators combine multiple Observable streams.

The four important ones are:

Operator	Trigger	Initial requirement	Completion behavior
<code>forkJoin</code>	All sources complete	Every source must complete	Emits final values once

Operator	Trigger	Initial requirement	Completion behavior
<code>combineLatest</code>	Any source emits	Every source must emit once	Continues until sources complete
<code>withLatestFrom</code>	Primary source emits	Secondary streams must emit once	Follows primary
<code>zip</code>	All sources have corresponding value	Values are matched by index	Continues while pairs are available

2.1 `forkJoin` — Final Result of All

Think:

`forkJoin` = `Promise.all()` for Observable completion

It waits until **all source Observables complete**, then emits their latest/final values once.

Example

```
import { forkJoin, of } from 'rxjs';
import { delay } from 'rxjs/operators';

const users$ = of(['Alice', 'Bob']).pipe(
  delay(1000)
);

const roles$ = of(['Admin', 'User']).pipe(
  delay(500)
);

forkJoin({
  users: users$,
  roles: roles$
}).subscribe(result => {
  console.log(result);
});
```

Output after both complete:

```
{
  users: ['Alice', 'Bob'],
  roles: ['Admin', 'User']
}
```

Important

This does **not** emit:

```
users → value
roles → value
users → value
```

It emits only once:

```
users  —————✓
roles  ————✓
      ↓
      forkJoin emits
```

Infinite Observable problem

This never emits:

```
forkJoin([
  timer(0, 1000),
  of('A')
]);
```

because:

```
timer(0, 1000)
```

doesn't complete.

Bound it:

```
import { forkJoin, timer, of } from 'rxjs';
import { take } from 'rxjs/operators';

forkJoin([
  timer(0, 1000).pipe(take(3)),
  of('A')
]).subscribe(console.log);
```

Use cases

- Load multiple APIs before displaying a page.
- Initialization data.
- Multiple independent HTTP requests.
- Final results from finite Observables.

Trade-offs

Advantages

- Simple all-at-once synchronization.
- Excellent for HTTP requests.
- Easy to reason about.

Disadvantages

- Requires completion.
- One error normally causes the combined Observable to error.
- Not suitable for continuously emitting streams.

2.2 combineLatest — Latest Combination

`combineLatest` emits whenever **any source emits**, but only after **every source has emitted at least once**.

Flow

A: —1—2—3—>

B: —X—Y—>

Output:

```
[1,X]
[2,X]
[2,Y]
[3,Y]
```

Example

```
import { combineLatest, interval } from 'rxjs';
import { map, take } from 'rxjs/operators';

const price$ = interval(1000).pipe(
  take(3),
  map(value => value + 100)
);

const quantity$ = interval(1500).pipe(
  take(2),
  map(value => value + 1)
);

combineLatest([
  price$,
  quantity$
]).subscribe(([price, quantity]) => {
  console.log({
    price,
    quantity,
    total: price * quantity
  });
});
```

Key rule

If:

```
A → emits
B → has never emitted
```

then:

`combineLatest` → no output

Once both have emitted:

```
A → new value → output
B → new value → output
A → new value → output
```

Use cases

- Reactive forms
- Filters
- Dashboard calculations
- UI state
- Derived values

Trade-offs

Advantages

- Excellent for continuously derived state.
- Always provides the latest known values.

Disadvantages

- Can produce many emissions.
- Every source must emit once.
- Can cause unnecessary recalculation if streams are noisy.

2.3 `withLatestFrom` — Primary Stream + Latest State

`withLatestFrom` has a **primary/source Observable**.

Only the primary Observable triggers output.

Flow

Primary: —A———B———C———>

Secondary: —X———Y———Z———>

Output:

[A,Y]

[B,Y]

[C,Z]

Example

```
import { interval } from 'rxjs';
import { withLatestFrom, map } from 'rxjs/operators';

const submit$ = interval(3000);

const formState$ = interval(1000).pipe(
  map(value => `Form state ${value}`)
);

submit$.pipe(
  withLatestFrom(formState$)
).subscribe(([submit, formState]) => {
  console.log(submit, formState);
});
```

The `submit$` stream controls when output occurs.

Use cases

- Button click + latest form state
- Save action + current application state
- Navigation event + current user
- Submit event + latest configuration

Trade-offs

Advantages

- Clearly defines a trigger stream.
- Prevents secondary emissions from triggering output.

Disadvantages

- Secondary stream must have emitted at least once.
- Easy to misuse when you actually need `combineLatest`.

2.4 `zip` — Pair by Position

`zip` matches values based on their sequence/index.

Example

```
import { zip, of } from 'rxjs';

zip(
  of(1, 2, 3),
  of('A', 'B', 'C', 'D')
).subscribe(result => {
  console.log(result);
});
```

Output:

```
[1, A]
[2, B]
[3, C]
```

The extra `D` has no corresponding value from the first Observable.

Flow

```
A: 1 — 2 — 3 —>
B: A — B — C —>
    ↓
[1,A]
[2,B]
[3,C]
```

Use cases

- Pairing corresponding values.
- Synchronizing sequences.
- Matching values by index.

Trade-offs

Advantages

- Deterministic positional pairing.
- Useful when streams represent corresponding sequences.

Disadvantages

- Can buffer values while waiting for another stream.
- Not suitable when you want "latest state".



3. Error Handling

Important operators:

```
catchError  
throwError  
retry
```

3.1 `catchError` — Recover From an Error

`catchError` intercepts an error and replaces the failed Observable with another Observable.

Example

```
import { of, throwError } from 'rxjs';  
import { catchError } from 'rxjs/operators';  
  
const api$ = throwError(  
  () => new Error('API Failed')  
);
```

```
api$.pipe(
  catchError(error => {
    console.error(error.message);

    return of({
      success: false,
      data: []
    });
  })
).subscribe(result => {
  console.log(result);
});
```

Output:

```
{
  success: false,
  data: []
}
```

Important concept

After an Observable errors:

```
next → next → next → ERROR
                        ↓
                    stream terminates
```

`catchError` lets you replace the terminated stream with another Observable.

3.2 `throwError` — Create an Error Observable

```
import { throwError } from 'rxjs';

const error$ = throwError(
  () => new Error('Something went wrong')
);
```

Useful when an operator needs to return an Observable that errors.

Example:

```
apiCall$.pipe(
  catchError(error => {
    if (error.status === 401) {
      return throwError(
        () => new Error('Unauthorized')
      );
    }
  })
  return of([]);
);
```

3.3 **retry** — Retry Failed Operations

Modern RxJS supports configuration-based retry.

```
import { retry } from 'rxjs/operators';

api$.pipe(
  retry({
    count: 3,
    delay: 1000
  })
);
```

Meaning:

```
Attempt 1 → fail
Attempt 2 → fail
Attempt 3 → fail
Attempt 4 → success
```

Maximum 3 retries

Retry with exponential backoff

```
import { timer } from 'rxjs';
import { retry } from 'rxjs/operators';
```

```
api$.pipe(  
  retry({  
    count: 3,  
    delay: (_error, retryCount) =>  
      timer(1000 * 2 ** (retryCount - 1))  
  })  
);
```

Timing:

```
Retry 1 → 1 sec  
Retry 2 → 2 sec  
Retry 3 → 4 sec
```

Trade-offs

Advantages

- Handles temporary failures.
- Useful for transient network errors.

Disadvantages

- Can increase server load.
- Should not blindly retry every error.
- Authentication/validation errors usually should not be retried.

3.4 retryWhen

`retryWhen` provides a custom notification-based retry mechanism.

For modern RxJS code, many common retry scenarios can be expressed more simply using:

```
retry({  
  count,  
  delay  
})
```

Therefore, for interview preparation:

Know `retryWhen` conceptually, but prefer the configurable `retry` API for straightforward retry/backoff implementations.

4. Multicasting

By default, many Observables are **unicast**.

Each subscriber gets its own subscription/execution.

Cold Observable



Two subscriptions
Two executions

With multicasting:



One **shared** execution

The most important operators here are:

```
share()  
shareReplay()
```

4.1 `share()` — Share Current Execution

`share()` shares a single subscription among active subscribers.

Example

```
import { timer } from 'rxjs';
import { share, tap } from 'rxjs/operators';

const source$ = timer(1000).pipe(
  tap(() => {
    console.log('Producer executed');
  })
);

const shared$ = source$.pipe(
  share()
);

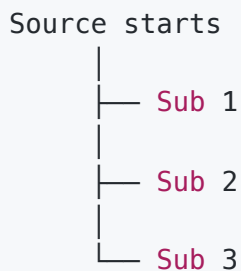
shared$.subscribe(value => {
  console.log('Subscriber 1:', value);
});

shared$.subscribe(value => {
  console.log('Subscriber 2:', value);
});
```

The source execution is shared between the subscribers.

Important timing detail

If subscribers arrive at different times:



All active **subscribers share** execution.

But a subscriber that arrives **after the shared stream has completed** does not automatically receive the previous value.

This is where `shareReplay` becomes useful.

4.2 shareReplay — Share + Replay

`shareReplay` shares execution while retaining previously emitted values for future subscribers.

Example

```
import { timer } from 'rxjs';
import { shareReplay, tap } from 'rxjs/operators';

const config$ = timer(1000).pipe(
  tap(() => {
    console.log('API called');
  }),
  shareReplay({
    bufferSize: 1,
    refCount: true
  })
);

config$.subscribe(value => {
  console.log('Subscriber 1:', value);
});

setTimeout(() => {
  config$.subscribe(value => {
    console.log('Subscriber 2:', value);
  });
}, 3000);
```

The second subscriber can receive the previously emitted value without requiring the source to execute again, subject to the configured lifecycle/reset behavior.

share VS shareReplay

Feature	<code>share()</code>	<code>shareReplay(1)</code>
Share execution	✓	✓
Replay previous value	✗	✓
Late subscriber gets previous value	✗	✓

Feature	<code>share()</code>	<code>shareReplay(1)</code>
Useful for caching	Sometimes	Often
Memory considerations	Lower	Higher
Typical use	Live shared stream	Cached state/config

Memory warning

Be careful with long-lived streams:

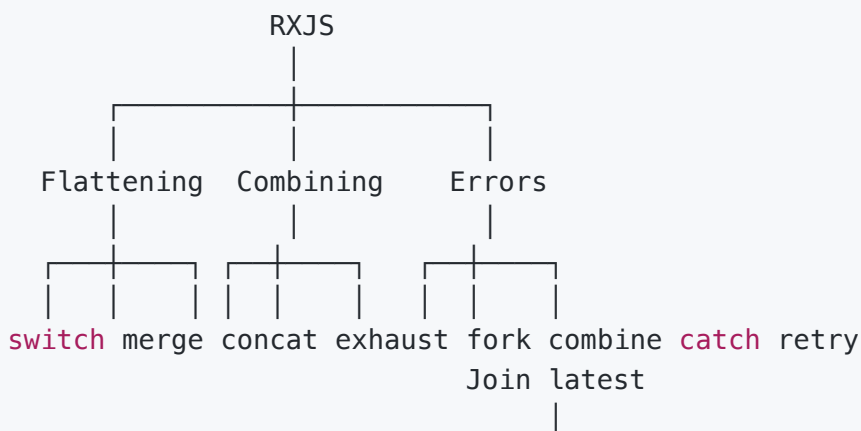
```
shareReplay({
  bufferSize: 1,
  refCount: false
})
```

Depending on the source and lifecycle, the shared subscription can remain alive longer than expected.

For Angular HTTP-style caching, a common pattern is:

```
user$ = this.http.get<User>('/api/user').pipe(
  shareReplay({
    bufferSize: 1,
    refCount: true
  })
);
```

Complete RxJS Operator Memory Sheet



withLatestFrom

zip

⚡ One-line revision

switchMap → Latest wins
mergeMap → Everything **concurrently**
concatMap → Everything sequentially
exhaustMap → First wins **while** busy

forkJoin → Final **value after ALL** complete
combineLatest → Latest **values** whenever **ANY** changes
withLatestFrom → **Primary** triggers + latest secondary
zip → Pair **values by index**

catchError → Recover/replace error
throwError → **Create** error
retry → Try again

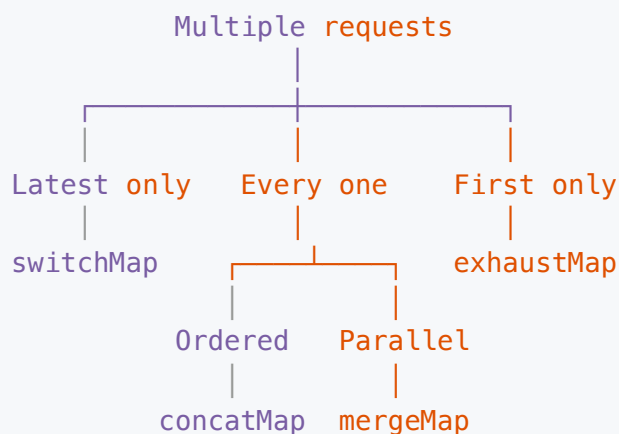
share → **Share** active execution
shareReplay → **Share** + replay previous **values**

🎯 The most important interview comparison

If the interviewer gives you:

"A user can trigger multiple API requests rapidly. What operator would you use?"

Don't answer immediately. Ask what the business requirement is:



This is the core decision framework to remember for RxJS interviews.